

Você é um especialista em arquitetura de sistemas de RAG (Retrieval-Augmented Generation) com profundo conhecimento em gestão de contexto de LLMs, processamento de diferentes tipos de documentos e otimização de custos de tokens.

Sua tarefa é produzir uma análise técnica detalhada e crítica sobre como implementar um pipeline de RAG robusto para uma base de conhecimento heterogênea. Esta análise servirá como documento de design para tomadas de decisão técnicas.

Análise por Tipo de Fonte de Dados

Para cada um dos seguintes tipos de fonte, você deve cobrir OBRIGATORIAMENTE:

1. **PDFs com tabelas**
2. **PDFs escaneados (OCR)**
3. **Wikis com links e estrutura hierárquica**
4. **Planilhas com fórmulas e dependências**

Para cada tipo, estruture sua resposta assim:

- **Desafio técnico para o pipeline RAG**: o que torna difícil extrair e indexar este tipo de dado? Que perda de informação ocorre?
- **Impacto na qualidade das respostas**: como esses desafios degradam a relevância, precisão ou utilidade das respostas ao usuário final?
- **Estratégia de tratamento recomendada**: que abordagens técnicas (parsing, normalização, chunking, metadata extraction) mitigam esses problemas?

Estimativa do Tamanho da Base em Tokens

Calcule o tamanho aproximado da base de conhecimento usando estes dados:

- ~800 documentos PDF com média de 10 páginas cada
- ~400 páginas de conteúdo wiki com média de 1.500 palavras cada
- ~50 planilhas

Use a regra prática de **0.75 palavras por token** para suas estimativas.

Seu cálculo deve deixar explícito:

- Tokens para PDFs (separar: conteúdo textual vs. tabelas vs. OCR)
- Tokens para wiki
- Tokens para planilhas
- Total estimado com margem de segurança (5-10%)

Análise de Orçamento de Contexto

Considerando os seguintes parâmetros:

- **Modelo**: GPT-4o com 128K tokens de janela de contexto
- **Overhead fixo**: system prompt + instruções consomem ~2.000 tokens
- **Tamanho de chunk padrão**: ~500 tokens por chunk

Você deve calcular e discutir:

- Quantos chunks cabem no orçamento restante por query?
- Qual o impacto dessa limitação na cobertura do conhecimento por pergunta?
- Como esse orçamento afeta a escolha entre estratégias de retrieval (top-K pequeno vs. grande, reranking, hybrid search)?
- Quais trade-offs emergem entre recuperar "mais documentos" vs. "contexto de qualidade melhor"?

Estratégia de Chunking Justificada

Recomende uma estratégia de chunking que considere:

- **Padrão de perguntas**: que tipos de perguntas você espera que os usuários façam? (ex: consultas simples de fato, comparações entre documentos, análises de tabelas, navegação em estruturas hierárquicas)

- **Problema do "Lost in the Middle"**: pesquisas mostram que informações no meio de contextos longos são frequentemente ignoradas pelos LLMs. Como sua estratégia de chunking e ordenação mitiga isso?
- **Heterogeneidade dos dados**: por que sua recomendação funciona bem para PDFs com tabelas, OCR, wikis e planilhas simultaneamente (ou se não funciona, o que muda por tipo)?

Justifique sua recomendação com evidência técnica, não apenas com boas práticas genéricas.

Revisão Crítica de Sua Própria Análise

Após completar a análise acima, você deve:

1. **Revisar sua própria análise** identificando:
 - Estimativas que podem ser otimistas demais (onde você pode ter subestimado complexidade ou tamanho)?
 - Pontos fracos em sua estratégia de chunking?
 - Riscos técnicos ou operacionais que você não considerou adequadamente?
 - Suposições não explícitas que poderiam falhar na prática?
2. **Documentar o feedback crítico** em uma seção separada chamada "Revisão Crítica e Riscos Não Mitigados"
3. **Atualizar sua análise original** incorporando os pontos de melhoria identificados na revisão.

Critérios de Qualidade

Sua análise deve ser:

- **Tecnicamente precisa**: use termos corretos, cite conceitos estabelecidos (lost in the middle, token budgets, retrieval strategies)
- **Crítica e realista**: não evite mencionar limitações, trade-offs ou cenários onde sua recomendação pode falhar
- **Justificada**: cada recomendação deve ter fundamento técnico claro, não ser genérica
- **Prática**: as recomendações devem ser implementáveis, com métricas e decisões concretas

Estruture o documento de forma clara e legível, com seções bem delimitadas e conclusões resumidas.

Estruturou análise técnica abrangente sobre arquitetura RAG heterogênea

Estruturou análise técnica abrangente sobre arquitetura RAG heterogênea

Excelente — li o documento da NovaTech com atenção. Ele serve como o "espelho real" da base de conhecimento: versões conflitantes, documentos informais sem governança, tabelas relacionais, lacunas de processo. Perfeito como base concreta para a análise.

Vou agora produzir o documento técnico completo.

Criou um arquivo, leu um arquivo

Criou um arquivo, leu um arquivo

O documento está completo. Aqui está um mapa dos pontos técnicos mais densos da análise — e onde as decisões realmente pesam:

O que o documento cobre (e por que cada seção importa)

Análise por tipo de fonte — o argumento central é que cada fonte degrada o RAG de forma diferente. PDFs tabulares falham no retrieval (chunk sem cabeçalho = vetor ruim), OCR

falha antes disso (no embedding — o texto corrompido nunca é encontrado), wikis falham na síntese (sem hierarquia, o modelo não entende escopo e exceções), planilhas falham na compreensão semântica (fórmulas têm similaridade zero com queries em linguagem natural).

Estimativa de tokens — o número que mais exige atenção é o das planilhas: faixa de 720K–3,0M tokens dependendo da complexidade real. É o componente com maior variância e maior chance de surpresa em produção. O total ficou em 5,3M tokens (baseline), com o pessimista em 7,5M.

Orçamento de contexto — o teto teórico de 248 chunks foi desmontado pela evidência do Lost in the Middle. O número operacional real é 10–20 chunks após reranking, cobrindo < 0,5% da base por query. Isso torna o reranker não um optional mas uma peça central da arquitetura.

Chunking hierárquico em 3 camadas — a justificativa não é genérica: Camada 1 (proposicional) → precisão para fact lookup; Camada 2 (semântica) → contexto procedimental; Camada 3 (documento-pai) → síntese cross-seção. As planilhas não têm Camada 3. Documentos OCR com baixa confiança não geram Camada 1.

Revisão crítica — identificou dois riscos que a análise original não resolveu adequadamente: (1) versões conflitantes (o caso PROC-042 v1 vs v2 não é resolvível só com metadados — precisa de um grafo de versões), e (2) propagação silenciosa de conteúdo não validado do FAQ informal sem sinalização ao usuário final.

Análise técnica rag pipeline
Documento · MD

Você é um arquiteto sênior de sistemas RAG com 10+ anos de experiência em pipelines de recuperação de informação em escala empresarial. Sua especialidade é identificar falhas de design, estimativas irrealistas e riscos operacionais ocultos.

****CONTEXTO:****

Acabei de produzir uma análise técnica sobre um sistema RAG que processará:

- ~800 PDFs (média 10 páginas cada)
- ~400 páginas wiki (média 1.500 palavras cada)
- ~50 planilhas
- Modelo alvo: GPT-4o (128K tokens de contexto)

****ANÁLISE ATUAL A REVISAR:****

- Título: Análise Técnica de Design: Pipeline RAG para Base de Conhecimento Heterogênea
- Arquivo: *analise-tecnica-rag-pipeline.md*
- Caminho: *.\Downloads\analise-tecnica-rag-pipeline.md*

****SUA MISSÃO:****

Revise criticamente esta análise identificando:

1. ****PONTOS FRACOS TÉCNICOS****

- Lacunas na estratégia de processamento por tipo de documento
- Falhas lógicas nas abordagens propostas
- Soluções simplistas para problemas complexos

2. ****ESTIMATIVAS OTIMISTAS****

- Cálculos de tokens que podem estar subestimados
- Taxas de conversão (palavras→tokens) que não consideram formatação/markup
- Pressupostos irrealistas sobre qualidade de OCR ou parsing

3. ****RISCOS NÃO CONSIDERADOS****

- Degradação de embeddings com chunks heterogêneos
- Edge cases (tabelas spanning páginas, fórmulas Excel complexas)
- Latência de retrieval com base de dados grande
- Manutenção e versionamento da base
- Qualidade inconsistente entre diferentes tipos de fonte

4. ****IMPACTO NO "LOST IN THE MIDDLE"*****

- A estratégia de chunking proposta realmente mitiga esse problema?
- Há risco de fragmentar informações críticas?

****FORMATO DE RESPOSTA:****

Pontos Fracos Identificados

[Liste 3-5 problemas críticos com explicação]

Estimativas a Revisar

[Indique quais números estão otimistas e valores mais realistas]

Riscos Adicionais

[Mínimo 4 riscos não mencionados na análise original]

Recomendações Corretivas

[Sugestões concretas para endereçar cada problema]

****Seja rigoroso e técnico. Prefira apontar problemas reais a validar otimismo.****

++++++Mostrar mais (Prompt em anexo no repositório)

Dissecou pipeline RAG identificando falhas técnicas, estimativas otimistas e riscos operacionais críticos

Dissecou pipeline RAG identificando falhas técnicas, estimativas otimistas e riscos operacionais críticos

Lido com atenção. Vou fazer uma revisão genuinamente rigorosa — identificando onde o documento é tecnicamente sólido, onde escorrega em otimismo, e onde tem lacunas que causariam problemas reais em produção.

Pontos Fracos Identificados

1. O fator de conversão palavras→tokens está incorreto para português técnico

O documento usa 0,72 palavras/token como ajuste conservador sobre o padrão inglês de 0,75. O problema é que esse ajuste ainda é otimista. O tokenizer cl100k_base (GPT-4) segmenta subpalavras por frequência em corpus predominantemente inglês. Palavras portuguesas com acentos e cedilha (características, multiplicadores, procedimento, vigência) e termos técnicos de domínio (CT-e, ANTT, roteirização) são consistentemente quebrados em mais tokens do que seus equivalentes em inglês. O valor realista para português corporativo com terminologia técnica é 0,60–0,65 palavras/token, não 0,72. Esse erro único eleva o total estimado em 10–20% antes de qualquer outra correção.

2. A estratégia de posicionamento "espiral" quebra documentos procedimentais

O documento recomenda posicionar os chunks com os mais relevantes nas extremidades do contexto (início e fim) para mitigar Lost in the Middle. Isso funciona para consultas factuais independentes, mas quebra ativamente documentos procedimentais sequenciais. Quando um usuário pergunta "quais são os passos para abrir um chamado de devolução?", os chunks corretos são as seções 3.3.1 a 3.3.5 do POL-001 — e elas precisam estar em ordem sequencial no contexto. O reranker os classifica por relevância e o posicionamento espiral embaralha a sequência. O LLM pode reconstruir a ordem por raciocínio, mas isso introduz fragilidade desnecessária e inconsistência entre modelos. A análise não faz distinção entre query types que requerem posicionamento por relevância versus posicionamento por ordem documental.

3. O mecanismo de construção do `doc_family_id` é assumido como resolvido, mas não é

O grafo de versões (seção 6.2) é a solução mais importante proposta para o problema PROC-042 v1/v2. Mas ele depende de que documentos da mesma família sejam detectados automaticamente. A análise não especifica o algoritmo de detecção. PROC-042 e PROC-042-v2 têm nomes similares — isso é o caso fácil. O caso difícil é "Tabela de Frete Nordeste 2023.xlsx" versus "NE_fretes_atualizado_nov23_FINAL_v2.xlsx" — ambos do mesmo grupo semântico, mas com nomes completamente diferentes. Sem um mecanismo explícito (fingerprinting de conteúdo, clustering semântico de títulos, ou atribuição manual), o grafo de versões não pode ser construído automaticamente de forma confiável. A análise trata isso como dado resolvido quando é, na prática, um problema de engenharia não trivial.

4. O overhead de contextualização por chunk está sistematicamente subestimado

A análise adiciona 8% de overhead para metadados. Mas o próprio documento recomenda incluir prefixos de breadcrumb em cada chunk:

[Contexto: PROC-042-v2, Seção 2.1, Tabela de Multiplicadores Regionais]

Esse prefixo sozinho consome 15–25 tokens. Para chunks de Layer 1 (80–150 tokens), isso representa 10–30% do tamanho do chunk apenas no prefixo. Adicionando os avisos de fonte para documentos informais (seção 6.3 recomenda ~35 tokens de aviso inline), e as anotações de links resolvidos em wikis, o overhead real de contextualização está mais próximo de 30–45% do tamanho do chunk base — não 8%. Isso tem impacto duplo: aumenta o tamanho total do índice e reduz o número de chunks que cabem no contexto de 128K tokens.

5. A estratégia de 3 camadas não define o mecanismo de escalada entre camadas

O documento descreve o que é cada camada mas não define o sinal que dispara a escalada de Layer 2 para Layer 3 (parent document). As opções são: (a) sempre usar Layer 3 quando encontrar um Layer 2 relevante (cara em tokens), (b) usar um limiar de confiança do reranker (como determinar o threshold?), ou (c) deixar o LLM decidir via tool call (adiciona uma LLM call extra com latência de 300–800ms). Cada opção tem trade-offs significativos que a análise não discute. Em produção, essa lacuna resultaria em decisões ad hoc de implementação que podem ser inconsistentes.

Estimativas a Revisar

Fator de conversão palavras→tokens

Parâmetro	Valor na análise	Valor realista	Impacto
Fator para português técnico no total de tokens	0,72 palavras/token	0,60–0,65 palavras/token	+10–20%
Overhead wiki (contextualização)	15%	35–42%*	+~230K tokens na wiki
Overhead tabelas serializadas tabulares	25–30%	40–55%**	+~200K tokens em PDFs
Overhead metadados por chunk	8% (global)	20–30% (pós-contextualização)	
Eleva todas as categorias			

* A estratégia recomendada (prefixo breadcrumb + snippets de links resolvidos) gera ~130 tokens de overhead em chunks de ~350 tokens = 37%.

** O formato de cabeçalho repetido por linha ([Contexto: X | Tabela: Y] | Coluna1 | Coluna2) em tabelas de mais de 20 linhas gera overhead de 40–55% sobre o conteúdo bruto.

Estimativa de tokens por planilhas: risco de underestimation de ordem de magnitude

A análise reconhece variância elevada (720K–3M tokens) mas ainda não aborda o caso crítico para logística. Uma planilha de fretes reais pode ter:

27 estados de origem × 27 destinos = 729 pares origem-destino

12 faixas de peso × 5 tipos de serviço = 60 combinações por par

Total: ~43.740 células ativas por planilha

Com serialização contextual de 8 palavras/célula: 350K palavras por planilha × 50 planilhas = 17,5M palavras → ~27M tokens (a 0,62 palavras/token). O cenário pessimista real para planilhas de logística não é 3M tokens — é potencialmente 20–30× maior. Isso não é edge case; é o caso típico de tabelas de precificação por CEP em transportadoras. A análise precisa de um inventário amostral das planilhas reais antes de qualquer estimativa.

Precisão do Tesseract para português

O benchmark citado (97% para documentos limpos) é medido em corpora ingleses. Para português com acentos em documentos escaneados de resolução padrão (~200 DPI), o baseline realista do Tesseract 4.x é 89–94%. Isso desloca toda a curva de degradação: o que a análise chama de "caso médio" (80–85%) é na prática o caso de documentos razoáveis, não ruins.

Latência do reranking: falta de especificação de hardware

A tabela de latências (seção 6.5) lista "150–400ms" para reranking com cross-encoder. Esse intervalo é válido para CPU inference com modelos médios. Com GPU (A10G ou equivalente), a latência cai para 10–50ms para 20 chunks. A diferença é arquitetural: em CPU, o reranking é o gargalo do pipeline; em GPU, não é. A decisão de hardware muda completamente a viabilidade do design para SLA de 5s.

Riscos Adicionais

Risco 1: Explosão do índice pela estratégia de 3 camadas — número total de chunks nunca calculado

O documento nunca computa o número total de chunks no índice, que é a métrica mais importante para sizing de infraestrutura. Estimativa:

Fonte	Layer 1 (chunks)	Layer 2 (chunks)	Total
800 PDFs × 10 páginas		$\sim 45 \text{ chunks/doc} \times 800 = 36.000$	$\sim 18 \text{ chunks/doc} \times 800 = 14.400$
400 páginas wiki	$\sim 12 \text{ chunks/pág} \times 400 = 4.800$		$\sim 5 \text{ chunks/pág} \times 400 = 2.000$
50 planilhas (est. baixo)	$\sim 80 \text{ chunks/planilha} \times 50 = 4.000$		$\sim 20 \times 50 = 1.000$
Total mínimo		$\sim 62.200 \text{ chunks}$	

Com 62K chunks e text-embedding-3-large (3.072 dimensões × float32 = 12KB/vetor): ~745MB apenas de vetores em RAM, sem considerar o índice HNSW (overhead adicional de 20–40%). O threshold de problemas de RAM começa em ~500MB para instâncias t3.medium (2GB RAM). O design precisa especificar o tier de infraestrutura.

Risco 2: Semantic drift do modelo de embedding invalida o índice sem aviso

Quando um novo modelo de embedding é lançado (OpenAI lançou text-embedding-3 substituindo ada-002 com incompatibilidade de espaço vetorial), vetores do índice antigo e vetores novos não são comparáveis na mesma busca. Não é possível adicionar novos documentos com o modelo novo sem reindexar toda a base. Para uma base em produção, isso significa:

Evento de re-embedding completo: downtime ou dual-index com custo 2× de armazenamento

Sem versão semântica de embeddings no design atual, o sistema degradará silenciosamente se qualquer parte do índice for construída com versões diferentes do modelo

O design precisa de versionamento explícito do modelo de embedding como campo nos metadados de cada chunk.

Risco 3: Histórico de conversa consome budget de retrieval em interações multi-turn

A análise calcula o budget como se cada query fosse independente. Em produção, um assistente de atendimento mantém histórico de conversa. Uma sessão típica de 6 turnos com 400 tokens por turno consome 2.400 tokens de histórico antes do primeiro chunk de retrieval. Em sessões longas (10+ turnos, comum em suporte), o histórico pode consumir 5.000–8.000 tokens, reduzindo o budget de 124K para ~116K tokens — o que não muda muito o número de chunks possíveis, mas viola a suposição de budget fixo que a análise usa. O problema real é quando o sistema precisa decidir o que fazer quando o histórico cresce: truncar o contexto de retrieval ou truncar o histórico? Nenhuma estratégia está definida.

Risco 4: Ausência completa de evaluation framework — impossível saber se o sistema funciona

O documento propõe um pipeline altamente sofisticado (3 camadas, reranking, grafo de versões, query expansion, HyDE) sem definir nenhuma métrica de avaliação ou metodologia de teste. Isso é o risco operacional mais subestimado de toda a análise. Sem um conjunto de Q&A de gold-standard (pelo menos 100–200 pares cobrindo os tipos de query esperados), é impossível responder:

O chunking hierárquico supera o chunking fixo de 512 tokens nesta base?

O reranker realmente melhora precision@10?

O grafo de versões reduz as inconsistências PROC-042 v1/v2?

A query expansion via HyDE melhora recall para queries ambíguas?

Cada uma dessas decisões arquiteturais pode ser invalidada por dados reais. O documento de design está incompleto sem um plano de avaliação que inclua: definição de métricas alvo (RAGAS, recall@K, MRR, faithfulness score), metodologia de criação de dataset de teste, e critério de sucesso mínimo para ir à produção.

Risco 5: Contradições internas entre documentos criam respostas "síntese" incorretas

O caso PROC-042 v1/v2 é mencionado como risco. Mas o impacto real é pior do que descrito. Considere: o FAQ (item 45) afirma que desconto automático se aplica "acima de 10 fretes/mês", mas PROC-042-v2 diz "a partir de 8 fretes/mês". Ambos os chunks serão recuperados para a query "quando tenho direito a desconto?". O LLM, sem instrução explícita de priorização, vai produzir uma resposta que pode: (a) citar o número errado, (b) citar ambos sem resolução, ou (c) calcular uma média implícita. O "AVISO DE FONTE" proposto apenas sinaliza a origem — não instrui o modelo sobre qual fonte priorizar quando há conflito direto de valor numérico. O system prompt precisa de uma regra explícita de hierarquia de confiança: documentos POL > documentos PROC > documentos SLA > FAQ informal.

Risco 6: O pipeline de OCR multi-motor proposto não tem critério de votação definido

A seção 1.2 propõe "votação de confiança por palavra" entre Tesseract e um motor comercial. Mas não define: o que fazer quando ambos discordam com confiança similar? Tesseract lê "1.4" e Azure Document Intelligence lê "1,4" (vírgula vs ponto — ambos com alta confiança)? Em documentos de logística com valores numéricos críticos, o critério de desempate não pode ser não-determinístico. Além disso, Azure Document Intelligence tem custo por página (~\$0,001–\$0,01/página dependendo da feature), e para 1.200 páginas OCR isso adiciona \$1,20–\$12 de custo de ingestão — aceitável, mas não mencionado no documento que citou custos de embedding.

Recomendações Corretivas

Para o fator de tokenização: refazer todos os cálculos com 0,62 palavras/token como baseline e apresentar o resultado como faixa [0,60; 0,65]. Conduzir amostragem real: pegar 10 documentos representativos de cada tipo, tokenizá-los com tiktoken (cl100k_base), e calcular o fator empírico antes de qualquer estimativa de produção.

Para o posicionamento espiral: substituir por uma estratégia condicional baseada no tipo de query. Classificar queries em factual (posicionamento espiral válido) vs procedural/sequential (preservar ordem documental dos chunks) vs comparative (intercalar chunks de fontes diferentes). O classificador pode ser uma chamada LLM barata com prompt de 1 linha antes do retrieval, adicionando ~100ms.

Para o doc_family_id: implementar detecção em duas fases. Fase 1: matching por prefixo normalizado de nome de documento (remover versão, datas, variações de capitalização). Fase 2: clustering semântico por embedding de título + abstract dos documentos para agrupar semanticamente similares. Marcar manualmente os conflitos detectados pelo sistema — não tentar automatizar a resolução de conflitos, apenas a detecção.

Para o overhead de contextualização: recalculer o tamanho do índice usando o overhead real de 30–45% e ajustar o número de chunks que cabem no contexto de 128K tokens. Com prefixos de 25 tokens em chunks de 350 tokens, o chunk efetivo é 375 tokens — reduzindo a capacidade de 248 para ~230 chunks teóricos (impacto pequeno no teto, mas relevante para o custo total do índice).

Para as planilhas: antes de qualquer estimativa, executar um inventário amostral: abrir 5 planilhas representativas, contar células ativas, e projetar o volume real. Se a base contiver planilhas de fretes por CEP, o volume pode ser 10–30× a estimativa atual. Considerar uma estratégia alternativa para planilhas muito grandes: em vez de indexar todas as células, indexar os metadados da planilha + sumário das dimensões e usar o arquivo original como tool_call externa (ex: code interpreter ou lookup via API), evitando a explosão de tokens.

Para o evaluation framework: criar um conjunto de 150 pares Q&A cobrindo: 50 fact lookups simples, 30 consultas procedimentais, 25 comparações entre versões de documentos, 25 queries com gaps documentais, 20 queries com informação contraditória. Definir métricas alvo: Recall@10 $\geq 0,85$, MRR $\geq 0,70$, faithfulness score (RAGAS) $\geq 0,80$ antes do deploy. Executar esse benchmark após cada mudança de pipeline para detectar regressões.

Para a hierarquia de confiança entre fontes: adicionar ao system prompt uma regra explícita e ordenada:

Hierarquia de fontes (em caso de conflito, usar a fonte de maior prioridade):

1. Documentos POL (políticas normativas)
2. Documentos PROC (procedimentos operacionais) — versão mais recente por vigência
3. Documentos SLA (compromissos contratuais)
4. FAQ-Atendimento: SEMPRE sinalizar como "prática informal não validada"

Em caso de conflito de valores numéricos entre fontes da mesma prioridade, reportar o conflito explicitamente ao usuário em vez de sintetizar um valor único.

Essa regra resolve programaticamente o caso PROC-042 v1/v2 e o conflito FAQ × PROC sem depender do grafo de versões para funcionar.

Síntese do diagnóstico: o documento tem valor como mapa de complexidade — identifica os problemas certos. As fragilidades mais sérias estão na quantificação (tokenização incorreta para português, overhead de contextualização subestimado, variância não endereçada de planilhas) e nos mecanismos não especificados (como construir o doc_family_id, como

escalar entre camadas de chunking, como resolver conflitos de valores numéricos entre fontes). O risco mais crítico e mais silencioso é a ausência de evaluation framework: um pipeline complexo sem métricas de validação pode ser pior que um chunking simples de 512 tokens — e sem testes, não há como saber.

****TAREFA:**** Gere um arquivo markdown estruturado chamado `consideracoes-revisao-rag.md` que sintetize:

1. ****Principais Problemas Identificados**** – Lista consolidada dos pontos críticos levantados na revisão
2. ****Ajustes de Estimativas**** – Comparação lado a lado entre valores originais e valores revisados (tokens, tamanho da base, chunks por query)
3. ****Riscos Priorizados**** – Riscos categorizados por impacto (Alto/Médio/Baixo) e probabilidade
4. ****Itens de Ação**** – Lista de próximos passos técnicos baseados no feedback (formato checklist)
5. ****Decisões Pendentes**** – Questões técnicas que ainda requerem prototipagem ou pesquisa adicional

****ENTRADA:**** Retorno da última revisão da análise

****FORMATO DE SAÍDA:**** - Use headers markdown apropriados (**##**, **###**) - Inclua tabelas para comparações numéricas - Use listas de checkbox - `[ ]` para itens de ação - Mantenha tom técnico e conciso - Máximo 800 palavras

++++++Mostrar mais (Promp em anexo no repositório)

Identificou excesso de palavras e reorganizou conteúdo estrategicamente
Identificou excesso de palavras e reorganizou conteúdo estrategicamente
O arquivo está com 1.072 palavras — acima do limite. Vou enxugar sem perder substância.

Otimizou estrutura de tabelas para reduzir contagem de palavras
Otimizou estrutura de tabelas para reduzir contagem de palavras
788 palavras — dentro do limite. Aqui está o mapa do que o documento contém e as escolhas de curadoria:

O que ficou e por quê:

A tabela de problemas foi reduzida para os 8 itens de maior impacto — P9 (OCR baseline para PT) e P10 (multi-turn) foram absorvidos nos itens de ação para evitar redundância. A tabela de riscos manteve os 4 reds e os 3 oranges — que são os que exigem decisão antes da próxima fase. P0 identifica os itens que bloqueiam produção independentemente do restante do design.

Decisão editorial mais importante: os itens de ação estão separados em três grupos com intenção deliberada. O grupo "Pré-ingestão" são bloqueantes reais — o pipeline não deve avançar sem eles. O grupo "Arquitetura" são mudanças de design que podem ser feitas em paralelo. O grupo "Validação" é o que transforma o documento de teoria em sistema confiável.

A linha final — "inventário de planilhas + tokenização empírica bloqueiam todas as estimativas de infraestrutura" — resume o ponto mais crítico da revisão inteira: os dois maiores riscos de underestimation têm solução barata e rápida (2–3 horas de trabalho), mas se ignorados podem invalidar toda a decisão de infraestrutura.